

AIN SHAMS UNIVERSITY

Faculty of Engineering

Computer Engineering and Software System

COURSE PROJECT Phase 1

CSE336: Software Design Patterns

Software Requirements Specification

Zakat Misr – Non-Profit Organization Management System

Prepared by:

Youssef Moustafa Elsayed	ID: 22P0047
Nour Eldeen Ahmed Rehab	ID: 22P0040
Mostafa Fouad Ahmed	ID: 22P0077
Ibrahim Shaker	ID: 22P0056
Abdelrahman Mostafa	ID: 22P0053
Mohamed Yasser Khallaf	ID: 22P0245

Presented to Dr. Raghda Essam



Contents

1 Software Requirement Specification (SRS) Document **1**
1.1 Introduction . . . 1
1.1.1 Glossary . . . 1
1.2 User Requirements . . . 1
1.3 Feasibility Analysis . . . 2
1.4 System Requirements . . . 2
1.4.1 Functional Requirements . . . 2
1.4.2 Non-Functional Requirements . . . 3
1.5 Client Requirements (Requirements Elicitation) . . . 4
1.6 Requirement Validation . . . 5
1.6.1 Requirements Traceability Matrix . . . 5
1.6.2 Source Traceability Matrix . . . 6
1.7 Anticipated Time Plan . . . 7
1.8 Use Case Modeling . . . 8
1.9 Conclusion . . . 10



List of Tables

1 Client Requirements 4
2 Requirements Traceability Matrix 5
3 Source Traceability Matrix 6
4 Anticipated Time Plan 7

List of Figures

1 Use-Case Diagram for the Zakat Misr System 9

1 Software Requirement Specification (SRS) Document

1.1 Introduction

The purpose of this document is to specify the software requirements for **Zakat Misr**, a Non-Profit Organization Management System. This Software Requirements Specification (SRS) details the intended behavior of the system, its features, and its constraints. It is the Phase 1 deliverable of the CSE336 course project.

The system is a centralized platform designed to streamline core operations for charitable institutions. Its primary goal is to automate donation processing, volunteer coordination, event planning, and beneficiary resource tracking. Phase 1 covers the requirements, the class diagram, the MVC structure, and the first six design patterns (Strategy, Observer, Decorator, Factory, Singleton, and MVC). The remaining patterns and modules are built in Phase 2.

1.1.1 Glossary

- **Zakat:** the obligatory annual charity in Islam, paid when a person's wealth exceeds the *nisab* threshold.
- **Sadaqah:** voluntary charity that can be given at any time and in any amount.
- **In-kind donation:** a donation of goods (food, clothes, supplies) rather than money.
- **Beneficiary:** a person or family that receives aid from the organization.

1.2 User Requirements

The software serves four primary user groups: Donors, Volunteers, Beneficiaries, and Administrators. Donors require a simple way to make contributions (Zakat, Sadaqah, or in-kind) and to view their receipts and contribution history. Volunteers require an interface to record their skills and availability, apply for tasks that match their skills, log hours, and obtain certificates. Beneficiaries require secure registration to submit needs, check eligibility, and track the aid they receive. Administrators require management screens to create and publish events, assign volunteers to tasks, manage users and lookup data, distribute donations to needs, and review program impact. Communication with all user groups must be supported through Email, SMS, and Social Media channels. The application must authenticate users against stored credentials and restrict each screen to the appropriate role.

1.3 Feasibility Analysis

- **Technical Feasibility:** The system is built in Java using the Swing toolkit for the user interface and the MVC architecture. All required tools are freely available. The data layer is accessed through a single database manager class, so a real relational database can replace the current in-memory storage without changing the rest of the system.
- **Economic Feasibility:** The project uses only free and open-source tools, so development and maintenance costs stay within the available resources.
- **Operational Feasibility:** The workflow mirrors real non-profit processes (donate, volunteer, attend events, receive aid), which makes adoption easy for staff, donors, and volunteers.
- **Legal and Ethical Feasibility:** Donor and beneficiary data are protected (passwords are stored hashed, and access is role-based), in line with data-privacy regulations.

1.4 System Requirements

1.4.1 Functional Requirements

- **1.1.** Users (Donors, Volunteers, Beneficiaries, Admins) should be able to create accounts and log in securely.
- **1.2.** Donors should be able to submit contributions (Zakat, Sadaqah, or in-kind; online or offline) and receive receipts, including tax-related documents for tax-deductible categories.
- **1.3.** Volunteers should be able to list their skills, set availability, and log service hours.
- **1.4.** Administrators should be able to create, schedule, publish, and cancel community fundraisers and events.
- **1.5.** Beneficiaries should be able to register needs and track aid distribution.
- **1.6.** Administrators should be able to assign volunteers to tasks based on automated skill matching.
- **1.7.** The system should dispatch automated event reminders and donation acknowledgments via Email, SMS, and Social Media.
- **1.8.** The system should generate beneficiary impact reports and volunteer certificates.



- **1.9.** Users should be able to register for open events and receive a ticket; administrators should be able to check attendees in.
- **1.10.** Users should be able to view and update their profile, change their password, and delete their account.
- **1.11.** Administrators should be able to manage users and the lookup tables (donation categories and skills) with create, read, update, and delete operations.

1.4.2 Non-Functional Requirements

- **2.1.** The system should respond to user actions in under 200 ms.
- **2.2.** The system should be available 24/7 with 99.9% uptime.
- **2.3.** The application should be able to serve at least 500 concurrent users.
- **2.4.** The software should run on any desktop operating system that supports Java (Windows, Linux, macOS).
- **2.5.** The system should scale to a growing number of records without redesign.
- **2.6.** The system should ensure secure user authentication and data protection (Role-Based Access Control).
- **2.7.** The system must be completed within the semester timeline.
- **2.8.** The system should use standard object-oriented design and design patterns.
- **2.9.** The application should follow the organization's privacy policies.
- **2.10.** The application should follow the country's laws and financial regulations regarding donations.

1.5 Client Requirements (Requirements Elicitation)

The basic requirements above were extended by collecting requirements from users of each group.

The following requirements were added on top of the basic requirements as a result:

ID	Client Requirement	Raised By	Realised By
CR-1	Donors can donate anonymously; the receipt hides their name.	Donors	Decorator pattern
CR-2	Donors can make a donation recurring (every n months).	Donors	Decorator pattern
CR-3	Each user chooses a preferred channel (Email, SMS, or Social Media) for notifications.	All users	Strategy pattern
CR-4	A beneficiary's eligibility is verified (income and family size) before aid is given.	Beneficiaries, Admins	Beneficiary service
CR-5	Volunteers receive a certificate for every 10 logged hours.	Volunteers	Volunteer entity

Table 1: Client Requirements

1.6 Requirement Validation

1.6.1 Requirements Traceability Matrix

Each row requirement depends on the column requirements marked with a check.

Req. ID	1.1	1.2	1.3	1.4	1.5	1.6	1.7	1.8	1.9	1.10	1.11
1.1											
1.2	✓										
1.3	✓										
1.4	✓										
1.5	✓	✓									
1.6	✓		✓	✓							
1.7	✓	✓		✓					✓		
1.8	✓		✓		✓						
1.9	✓			✓							
1.10	✓										
1.11	✓										

Table 2: Requirements Traceability Matrix

1.6.2 Source Traceability Matrix

Req. ID	Donors	Volunteers	Beneficiaries	Admins	Course Team
1.1	✓	✓	✓	✓	
1.2	✓			✓	
1.3		✓		✓	
1.4				✓	
1.5			✓	✓	
1.6				✓	
1.7	✓	✓	✓	✓	
1.8		✓	✓	✓	
1.9	✓	✓	✓	✓	
1.10	✓	✓	✓	✓	
1.11				✓	
2.1	✓	✓	✓	✓	
2.2	✓	✓	✓	✓	
2.3	✓	✓	✓	✓	
2.4	✓	✓	✓	✓	
2.5				✓	
2.6	✓	✓	✓	✓	
2.7					✓
2.8					✓
2.9				✓	
2.10	✓	✓	✓	✓	

Table 3: Source Traceability Matrix

1.7 Anticipated Time Plan

The project formally started in the last week of October with requirements gathering and the class diagram. Phase 1 implementation (MVC and the first six patterns) took place in the first week of November, and the second week of November is reserved for midterm exams. The Phase 2 patterns and core modules are built from the third week of November, followed by system integration and testing before the final oral discussion in mid-December.

Month	**Oct**	**November**				**December**	
Weeks	**4**	**1**	**2**	**3**	**4**	**1**	**2**
1. Requirements Gathering & Class Diagram	■						
2. MVC & Phase 1 Patterns Implementation		■					
3. Phase 2 Patterns & Core Modules				■	■	■	
4. System Integration & Testing							■

Table 4: Anticipated Time Plan

1.8 Use Case Modeling

To visualize the interactions between the system actors (Volunteers, Donors, Beneficiaries, Administrators, and the Notification Service) and the core functionalities of the system, the Use Case Diagram is presented below. It shows the system boundary, the include relationships (such as credential validation and notification dispatching), and the role-specific flows.

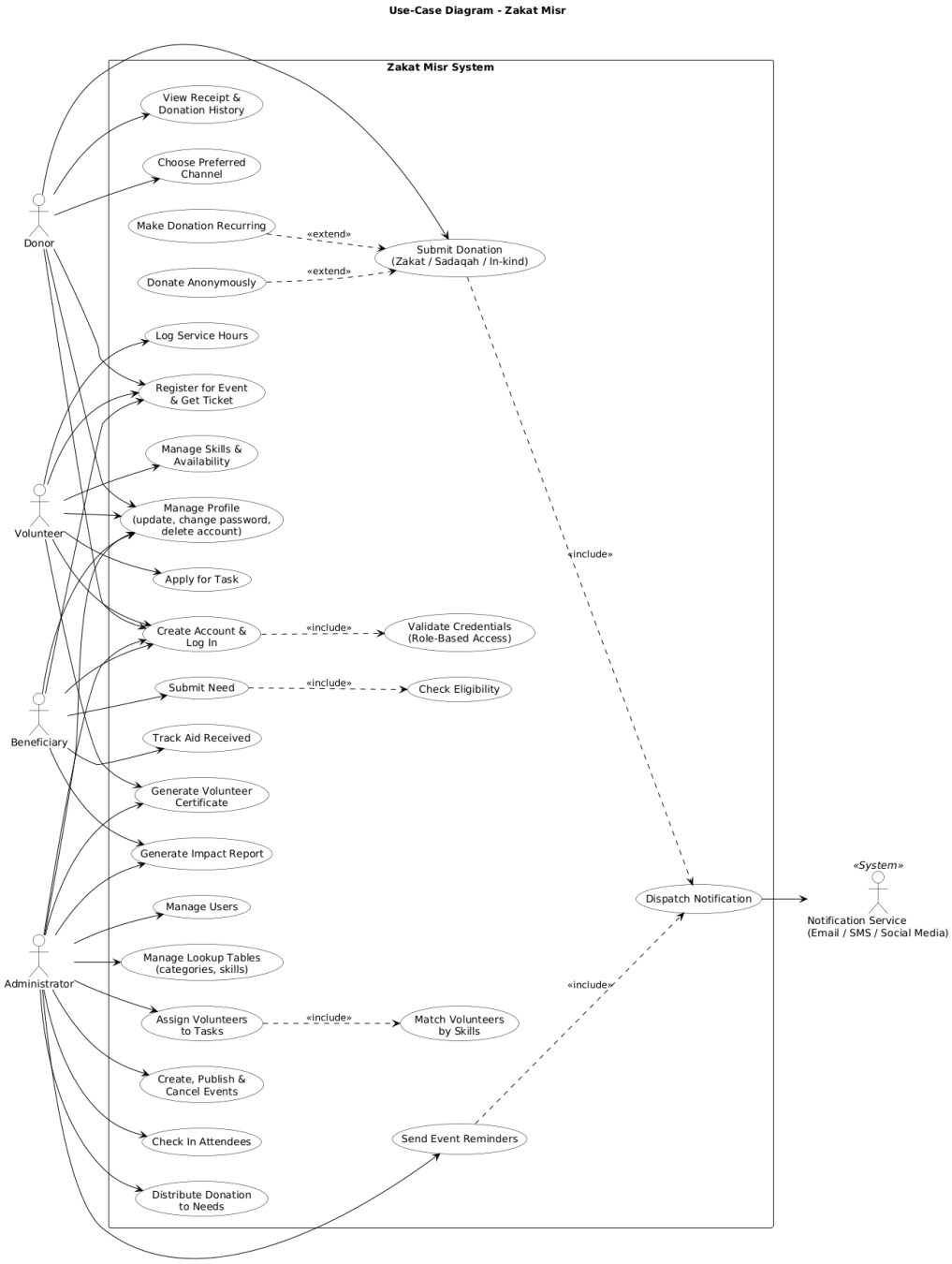


Figure 1: Use-Case Diagram for the Zakat Misr System



1.9 Conclusion

Zakat Misr addresses the operational challenges of charitable organizations by providing one secure platform for donation tracking, volunteer coordination, event hosting, and beneficiary aid distribution. By defining clear requirements, extending them with client input, and maintaining strict traceability, this SRS gives a solid baseline for the Phase 1 design (class diagram, MVC, and the Strategy, Observer, Decorator, Factory, and Singleton patterns) and for the Phase 2 work that follows.